MACHINE LEARNING ENGINEERING

Hands-On Lab Series 01

Five End-to-End Machine Learning Exercises

Implementation, Code Walkthrough, Evaluation Results,
Visual Diagnostics, and Business Interpretation

Submitted by: Pendli Harshita
Designation: Data Engineering Trainee

Programming language: Python
Libraries: Pandas, NumPy, Matplotlib, Scikit-learn
Notebook: `ML_Handson_Assignment.ipynb`
Reference document: Machine Learning Engineering Lab Guide
Report: Five practical exercises

Contents

1 Introduction and Laboratory Overview **2**
1.1 Exercises at a Glance . . . 2
1.2 Software and General Method . . . 2

2 Exercise 1: Multiple Linear Regression – Insurance Cost Prediction **2**
2.1 Problem Statement . . . 2
2.2 Dataset Description and Initial Inspection . . . 2
2.3 Code Walkthrough: Data Preparation and Pipeline . . . 3
2.4 Evaluation Metrics and Recorded Results . . . 3
2.5 Coefficient Interpretation . . . 4
2.6 Residual Diagnostics and Interpretation . . . 4
2.7 Business Interpretation . . . 5

3 Exercise 2: Logistic Regression – Bank Marketing Conversion **5**
3.1 Problem Statement and Dataset . . . 5
3.2 Code Walkthrough: Encoding, Preprocessing, and Training . . . 5
3.3 Model Evaluation at Two Thresholds . . . 5
3.4 Confusion Matrix and Threshold Trade-Off . . . 6
3.5 ROC and Precision–Recall Diagnostics . . . 6
3.6 Business Interpretation . . . 6

4 Exercise 3: Decision Trees – Interpretable Classification **7**
4.1 Objective and Dataset . . . 7
4.2 How the Code Works . . . 7
4.3 Recorded Model Comparison . . . 8
4.4 Confusion Matrix and Model Interpretation . . . 8
4.5 Practical Interpretation . . . 8

5 Exercise 4: Random Forest – Ensemble Learning and Feature Importance **8**
5.1 Objective and Dataset . . . 8
5.2 Baseline Forest and OOB Validation . . . 9
5.3 Hyperparameter Tuning . . . 9
5.4 Tuned Model Results . . . 9
5.5 Tuned Random Forest Confusion Matrix . . . 10
5.6 MDI Feature Importance . . . 10
5.7 Permutation Feature Importance . . . 11
5.8 Classifier Benchmarking . . . 12
5.9 Business Interpretation . . . 13

6 Exercise 5: K-Means – Wholesale Customer Spending Personas **13**
6.1 Objective and Dataset . . . 13
6.2 Code Walkthrough: Log Transformation and Scaling . . . 14
6.3 Elbow Method . . . 14
6.4 Silhouette Analysis . . . 15
6.5 Four-Cluster Spending Profiles . . . 16
6.6 Customer Counts and Business Personas . . . 18
6.7 Actionable Recommendations . . . 19
6.8 Interpretation . . . 19

7 Cross-Exercise Comparison and Reproducibility **19**
7.1 Consolidated Results . . . 19
7.2 Saved Artifacts and Reproducibility . . . 20

8 Conclusion **20**

1 Introduction and Laboratory Overview

This report documents the implementation and recorded outputs of five machine-learning exercises from the supplied notebook and laboratory guide. The work covers supervised learning for continuous-value prediction, binary classification, interpretable decision trees, ensemble classification, and unsupervised customer segmentation.

Each exercise follows a consistent practical workflow: load and inspect the dataset, identify the target and predictors, prepare the data, train the model, evaluate it using suitable metrics, interpret diagnostic plots, and explain how the results can inform a business decision. The notebook also saves prediction tables and visual outputs for use in the final submission.

1.1 Exercises at a Glance

No.	Technique	Dataset / Problem	Main evaluation
1	Multiple Linear Regression	Medical insurance charges	MSE, RMSE, MAE, R^2, coefficients, residual diagnostics
2	Logistic Regression	Bank term-deposit marketing	ROC-AUC, precision, recall, F1, threshold comparison, confusion matrix
3	Decision Tree	Breast Cancer Wisconsin Diagnostic data in the notebook	Accuracy, precision, recall, F1, tree depth, pruning
4	Random Forest	Bank Marketing dataset	OOB score, test metrics, GridSearchCV, MDI and permutation importance
5	K-Means Clustering	Wholesale customer annual spending	Inertia, silhouette score, cluster centroids and personas

1.2 Software and General Method

The notebook uses Python with Pandas for tabular data, NumPy for numerical operations, Matplotlib and related plotting utilities for diagnostics, and Scikit-learn for preprocessing, modelling, evaluation, and clustering. The notebook obtains the Bank Marketing and Wholesale Customers datasets from UCI data sources. The recorded dataset sizes and scores below are taken from the notebook outputs.

2 Exercise 1: Multiple Linear Regression – Insurance Cost Prediction

2.1 Problem Statement

The objective is to estimate an individual's annual medical insurance charges using demographic and lifestyle attributes. A linear regression model learns a weighted relationship between the input variables and the continuous target `charges`. The output is a dollar-valued prediction, so both the size of the prediction error and the amount of target variance explained are important.

2.2 Dataset Description and Initial Inspection

The Medical Cost Personal Data dataset contains 1,338 rows and seven columns. The six predictors are `age`, `sex`, `bmi`, `children`, `smoker`, and `region`; `charges` is the target. All seven columns have 1,338 non-null values. The data check reported one duplicate row.

Table 2: Descriptive statistics reported by the notebook

Variable	Mean	Std. dev.	Minimum	Median	Maximum
Age	39.21	14.05	18	39	64
BMI	30.66	6.10	15.96	30.40	53.13
Children	1.09	1.21	0	1	5
Charges ($)	13,270.42	12,110.01	1,121.87	9,382.03	63,770.43

The target skewness is 1.5159, showing a right-skewed charge distribution: most observations are concentrated at lower or moderate costs, while a smaller number of individuals have much higher charges. This motivated comparing the raw target with a $\log(1 + x)$ target transformation.

2.3 Code Walkthrough: Data Preparation and Pipeline

The notebook first loads the CSV with Pandas, checks the shape, previews records, inspects data types and missing values, and computes descriptive statistics and target skewness. The feature matrix X contains the six predictors and the target vector y contains `charges`.

The data is split into 1,070 training rows and 268 test rows, using an 80/20 split and a fixed random state of 42. The preprocessing is kept inside a Scikit-learn pipeline so that the transformations are fitted on the training data and then applied to the test data consistently.

- **Numeric features:** median imputation followed by `StandardScaler` for age, BMI, and number of children.
- **Categorical features:** most-frequent-value imputation followed by one-hot encoding. The first encoded category is dropped to reduce redundant dummy columns.
- **Estimator:** `LinearRegression` is fitted after preprocessing.

Listing 1: Core structure of the regression pipeline

```
num_cols = ['age', 'bmi', 'children']
cat_cols = ['sex', 'smoker', 'region']

preprocessor = ColumnTransformer(transformers=[
    ('num', numeric_pipeline, num_cols),
    ('cat', categorical_pipeline, cat_cols)
])

model_pipeline = Pipeline(steps=[
    ('prep', preprocessor),
    ('regressor', LinearRegression())
])
model_pipeline.fit(X_train, y_train)
predictions = model_pipeline.predict(X_test)
```

The model then predicts charges for the held-out test set. The notebook also saves a table of actual and predicted charges so that individual errors can be reviewed, rather than relying only on a single summary score.

2.4 Evaluation Metrics and Recorded Results

Mean Squared Error (MSE) averages squared prediction errors; RMSE is its square root and is expressed in the same dollar units as the target. MAE is the average absolute error and is less dominated by very

large errors than squared-error metrics. The R^2 score measures the proportion of test-target variance explained relative to a constant-mean baseline.

Table 3: Raw-target and log-target regression comparison

Metric	**Raw charges**	$\log(1+$ **charges**$)$ **target**
MSE	33,596,915.85	–
RMSE	\$5,796.28	\$7,814.06
MAE	\$4,181.19	\$3,888.44
R^2	0.7836	0.6067

The raw-target model explains 78.36% of the observed test variation according to R^2. It has lower RMSE than the log-target model, whereas the log-target model has lower MAE. Thus, the log transformation reduced average absolute error in this run but did not improve RMSE or R^2.

2.5 Coefficient Interpretation

The fitted raw-target model reported an intercept of \$8,955.24. The estimated coefficients are listed below. Numeric-feature coefficients refer to one standardized unit because the numeric columns were standardized; categorical coefficients refer to the encoded category relative to its reference category.

Table 4: Regression coefficients reported by the pipeline

Feature	**Coefficient**
Smoker: yes	\$23,651.13
Age (standardized)	\$3,614.98
BMI (standardized)	\$2,036.23
Children (standardized)	\$516.89
Sex: male	−\$18.59
Region: northwest	−\$370.68
Region: southeast	−\$657.86
Region: southwest	−\$809.80

The positive smoker coefficient indicates a large adjusted association between smoking status and predicted charges in this linear model, holding the other included variables constant. The coefficient reported by the notebook is \$23,651.13.

2.6 Residual Diagnostics and Interpretation

Residuals are calculated as actual target values minus model predictions. A plot of residuals against fitted values is used to check whether errors are randomly distributed around zero and whether their spread changes across the prediction range. A residual distribution plot helps reveal skewness and unusually large errors. An actual-versus-predicted plot shows how closely predictions follow the ideal diagonal relationship.

The notebook saves the following diagnostic figures: `02_actual_vs_predicted.png`, `03_residuals_vs_fitted.png` and `04_residual_distribution.png`. It also exports the test prediction table. The diagnostic interpretation recorded in the notebook notes that a fan-shaped residual pattern suggests non-constant error variance, particularly at higher expense levels.

2.7 Business Interpretation

The model can provide a baseline estimate of medical charges from the available customer attributes. The strong positive smoker coefficient highlights smoking status as a major predictor in this dataset. For practical use, predictions should be accompanied by an explanation of uncertainty, because a single linear model cannot capture every factor affecting an individual's medical expenses. Residual analysis is therefore important before using predictions to support operational decisions.

3 Exercise 2: Logistic Regression – Bank Marketing Conversion

3.1 Problem Statement and Dataset

The objective is to predict whether a customer subscribed to a term deposit following a direct-marketing campaign. The target column `y` contains `yes` and `no`. The UCI Bank Marketing dataset contains 45,211 rows and 17 columns.

The target distribution is imbalanced: 39,922 customers (88.3%) did not subscribe and 5,289 customers (11.7%) did subscribe. This imbalance means accuracy alone can be misleading. Recall measures the proportion of actual subscribers found by the model, while precision measures the proportion of customers predicted as subscribers who actually subscribed.

3.2 Code Walkthrough: Encoding, Preprocessing, and Training

The notebook downloads the UCI archive and saves a local copy as `bank-full.csv`. It separates numeric features (age, balance, day, duration, campaign, pdays, previous) from categorical features (job, marital, education, default, housing, loan, contact, month, poutcome). The target is encoded as a binary variable.

A stratified train/test split preserves the approximate class proportions in both partitions, producing 33,908 training records and 11,303 test records. Numeric features use robust scaling to reduce the influence of extreme values, while categorical variables are one-hot encoded. The classifier uses class weighting to account for the minority positive class and is trained with a maximum of 1,000 iterations.

Listing 2: Core logistic-regression training and thresholding steps

```
log_reg = LogisticRegression(
    class_weight='balanced', max_iter=1000,
    random_state=42
)
log_reg.fit(X_train_prepared, y_train)
probabilities = log_reg.predict_proba(X_test_prepared)[:, 1]

# Convert probabilities to class predictions using threshold 0.35
custom_predictions = (probabilities >= 0.35).astype(int)
roc_auc = roc_auc_score(y_test, probabilities)
```

The model produces a probability for the positive class. A threshold converts that probability into a final class label. Lowering the threshold tends to increase positive predictions and recall, while usually reducing precision.

3.3 Model Evaluation at Two Thresholds

The notebook reports ROC-AUC = 0.9085. ROC-AUC measures how well the model ranks positive cases above negative cases over all possible thresholds. The classification report was calculated at both 0.50 and 0.35.

Table 5: Logistic regression results at the two evaluated thresholds

Metric	Threshold 0.50	Threshold 0.35
Accuracy	0.8481	0.7785
Precision (subscribed)	0.4226	0.3343
Recall (subscribed)	0.8162	0.9017
F1-score (subscribed)	0.5569	0.4877
ROC-AUC	0.9085	0.9085

The threshold of 0.35 increases positive-class recall from 0.8162 to 0.9017, while precision decreases from 0.4226 to 0.3343. Accuracy also decreases because more customers are classified as likely subscribers. This is the intended trade-off when the business priority is to avoid missing potential subscribers.

3.4 Confusion Matrix and Threshold Trade-Off

At threshold 0.35, the confusion matrix is

$$\begin{bmatrix} 7607 & 2374 \\ 130 & 1192 \end{bmatrix}$$

where the first row corresponds to actual non-subscribers and the second row corresponds to actual subscribers. Therefore, the model correctly identifies 7,607 non-subscribers and 1,192 subscribers. It incorrectly flags 2,374 non-subscribers as potential subscribers and misses 130 actual subscribers.

Table 6: Confusion-matrix interpretation at threshold 0.35

Outcome	Business meaning
True negative (7,607)	Non-subscribers correctly deprioritized for the campaign.
False positive (2,374)	Non-subscribers who may receive outreach, using agent time without a resulting subscription.
False negative (130)	Actual subscribers missed by the model; these represent potential lost conversion opportunities.
True positive (1,192)	Actual subscribers correctly identified for prioritization.

The notebook also evaluates thresholds from 0.10 to 0.90. For example, at threshold 0.65, accuracy is 0.8819, precision is 0.4966, recall is 0.7088, and F1 is 0.5840. This illustrates that the threshold should be selected according to the cost of missed subscribers versus unnecessary calls, rather than chosen solely to maximize accuracy.

3.5 ROC and Precision–Recall Diagnostics

The ROC curve visualizes the true-positive rate against the false-positive rate across thresholds. The precision–recall analysis is especially relevant for an imbalanced target because it focuses on the positive class. Together with the confusion matrix, these diagnostics provide a more complete view than accuracy alone.

3.6 Business Interpretation

The model can be used to rank leads for bank marketing campaigns. A lower threshold is suitable when the campaign seeks to capture as many potential subscribers as possible and can tolerate extra

outreach. A higher threshold is preferable when sales-agent capacity is limited and the bank wants to prioritize leads with a higher predicted likelihood. Threshold choice should therefore be aligned with campaign costs and expected value per subscription.

4 Exercise 3: Decision Trees – Interpretable Classification

4.1 Objective and Dataset

This exercise investigates decision trees as interpretable classification models. Trees make predictions through a sequence of feature-based conditions, which can be translated into readable decision rules. The notebook implementation uses the Breast Cancer Wisconsin Diagnostic dataset, containing 569 records and 30 input features. The target classes are malignant (212 records) and benign (357 records).

The dataset is divided into 455 training examples and 114 test examples. The exercise compares Gini impurity and entropy, demonstrates the behaviour of an unrestricted tree, constrains model complexity, and applies cost-complexity pruning.

4.2 How the Code Works

The notebook loads the dataset, checks the dimensions and class distribution, and separates the predictors from the diagnosis target. It fits a decision tree using Gini impurity and another using entropy. Both criteria evaluate the quality of candidate splits; they use different mathematical measures, but both aim to create child nodes that are more class-pure than the parent node.

An unrestricted tree is fitted to demonstrate how a deep tree can memorize training observations. The notebook then compares constrained trees using maximum depth and minimum-sample settings. Cost-complexity pruning uses the pruning path to evaluate a sequence of complexity penalties, represented by α. As α increases, additional branches are removed and the tree becomes simpler.

Listing 3: Decision-tree fitting and readable rule extraction

```
dt_clf = DecisionTreeClassifier(
    criterion='gini', max_depth=4,
    min_samples_leaf=15, random_state=42
)
dt_clf.fit(X_train_prepared, y_train)

rules = export_text(
    dt_clf, feature_names=feature_names, max_depth=3
)
print(rules)
```

The `export_text` utility prints the learned split conditions in a human-readable format. Each path from the root to a leaf forms a sequence of conditions leading to a class prediction.

4.3 Recorded Model Comparison

Table 7: Decision-tree training and test performance

Configuration	Train acc.	Test acc.	Precision	Recall	F1	Depth
Unrestricted – Gini	1.0000	0.9123	0.9559	0.9028	0.9286	7
Unrestricted – Entropy	1.0000	0.9123	0.9697	0.8889	0.9275	6
Pruned – Gini	0.9714	0.9386	0.9577	0.9444	0.9510	3
Maximum depth 5	0.9934	0.9211	0.9565	0.9167	0.9362	5
Best cross-validated tree	0.9868	0.9386	0.9577	0.9444	0.9510	4

The selected pruned model reported test accuracy 0.9386, precision 0.9577, recall 0.9444, and F1-score 0.9510. Its depth was 3 and it contained five leaves. The reported pruning parameter was `ccp_alpha` = 0.007641125997290381. The grid-search configuration reported a best cross-validation accuracy of 0.9385.

4.4 Confusion Matrix and Model Interpretation

The recorded confusion matrix, using class order malignant followed by benign, was

$$\begin{bmatrix} 39 & 3 \\ 4 & 68 \end{bmatrix}.$$

This means 39 malignant cases and 68 benign cases were correctly classified, while three malignant cases and four benign cases were misclassified according to this matrix's row/column order.

The unrestricted trees achieve perfect training accuracy but lower test accuracy, a pattern consistent with overfitting. Pruning reduces the number of branches and improves the recorded test score in this run. A shallower tree is also easier to inspect and explain to a non-technical stakeholder. The rules printed by the notebook provide the basis for communicating how individual feature thresholds contribute to a prediction.

4.5 Practical Interpretation

A decision tree offers transparency: a prediction can be traced through explicit feature conditions rather than described only through model weights. In a diagnostic setting, such rules can support review and explanation, but they should not replace clinical expertise or independent validation. The notebook's measured results describe this dataset and split; they do not, by themselves, establish clinical suitability.

5 Exercise 4: Random Forest – Ensemble Learning and Feature Importance

5.1 Objective and Dataset

The Random Forest exercise returns to the Bank Marketing dataset and explores an ensemble of decision trees. Random Forest trains trees on bootstrap samples and considers a random subset of features at each split. Aggregating many trees can reduce the variance of a single decision tree and often improves generalization.

The notebook loads 45,211 records with 17 columns and a binary target. There are 39,922 non-subscribers and 5,289 subscribers. The preprocessing separates numeric and categorical columns,

applies suitable transformations, and fits the classifier within a pipeline. The notebook evaluates an initial forest, tracks out-of-bag (OOB) performance, tunes hyperparameters with GridSearchCV, calculates two feature-importance measures, and compares three classifiers.

5.2 Baseline Forest and OOB Validation

The baseline Random Forest uses bootstrap sampling and OOB scoring. A record not selected in a particular tree's bootstrap sample can be predicted by that tree as an out-of-bag observation. Aggregating those predictions gives an internal generalization estimate without requiring a separate validation fold for every tree.

The recorded baseline OOB score was 0.9043, OOB error was 0.0957, and test accuracy was 0.9065. The tuned model reported an OOB score of 0.9005. The convergence values recorded as the number of trees increased were 0.8981 at 25 trees, 0.9029 at 50, 0.9037 at 75, 0.9043 at 100, 0.9053 at 150, and 0.9056 at 200 trees. The sequence shows that the OOB estimate changes less as additional trees are added.

5.3 Hyperparameter Tuning

The notebook uses GridSearchCV with three-fold cross-validation and F1 scoring. It searches the following combinations:

- Number of estimators: 100 and 200.
- Maximum features considered at a split: `sqrt` and `log2`.
- Maximum tree depth: 8 and 12.

This produces eight parameter combinations and 24 model fits across the three folds. The selected configuration was 200 estimators, `max_features=sqrt`, and maximum depth 12. The best cross-validation F1 was 0.3539.

5.4 Tuned Model Results

Table 8: Recorded baseline and tuned Random Forest metrics

Metric	Baseline forest	Tuned forest
OOB score	0.9043	0.9005
OOB error	0.0957	0.0995
Test accuracy	0.9065	0.9014
Test precision	–	0.7374
Test recall	–	0.2443
Test F1-score	0.5002	0.3670
Test ROC-AUC	–	0.9243

The tuned model has high test accuracy and ROC-AUC, but its positive-class recall is 0.2443. With an imbalanced target, accuracy can remain high when most observations belong to the negative class. Precision, recall, F1, and the confusion matrix therefore need to be read together to understand how the classifier handles subscribers.

5.5 Tuned Random Forest Confusion Matrix

The following screenshot is positioned here because it supports the explanation of the tuned model's test predictions. The horizontal axis shows the predicted label and the vertical axis shows the true label.

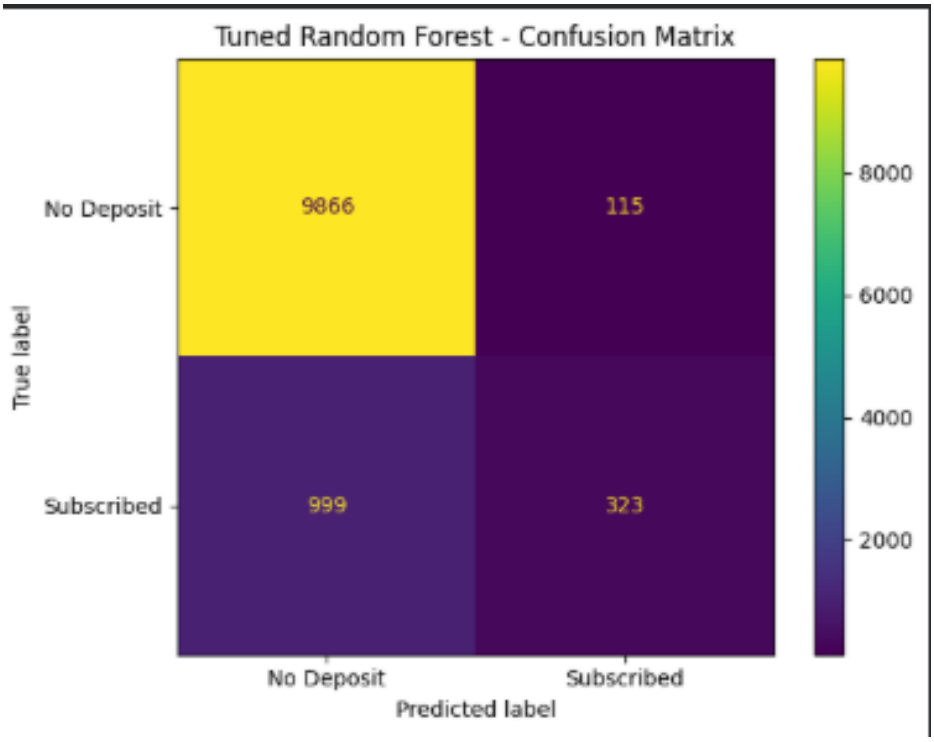


Figure 1: Tuned Random Forest confusion matrix. The displayed counts are 9,866 true negatives, 115 false positives, 999 false negatives, and 323 true positives.

The matrix contains 9,866 non-subscribers correctly predicted as "No Deposit", 115 non-subscribers incorrectly predicted as "Subscribed", 999 subscribers missed by the model, and 323 subscribers correctly identified. The high number of false negatives explains why positive-class recall is low even though overall accuracy is approximately 90%.

5.6 MDI Feature Importance

Mean Decrease in Impurity (MDI) is calculated from the reduction in node impurity contributed by each feature across the fitted trees, aggregated and normalized. The notebook's top MDI features were `duration` (0.34330), `poutcome` (0.13731), `month` (0.11209), `age` (0.06352), and `balance` (0.04592). Call duration is the largest feature by this measure.

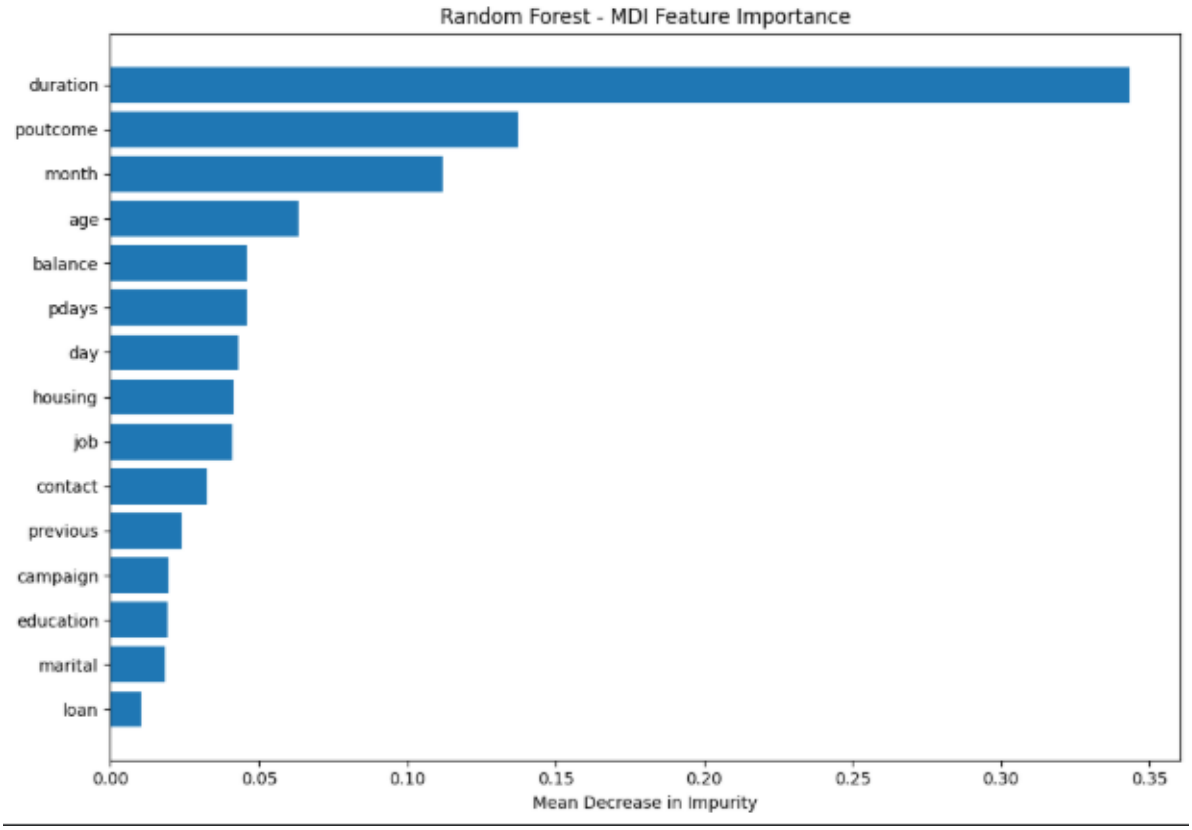


Figure 2: Random Forest MDI feature importance ranking.

The bar chart provides a relative ranking of features used by the fitted trees. MDI can favour variables with more possible split points or categories, so it is useful to compare it with a second method rather than treating it as a definitive causal explanation.

5.7 Permutation Feature Importance

Permutation importance measures the change in a chosen evaluation metric when one feature's values are shuffled. If the metric drops substantially, the model depends on that feature for its predictive performance. In the notebook, the evaluation metric is test F1-score.

The top permutation results were `poutcome` (mean decrease in F1 = 0.17113, standard deviation 0.00411), `duration` (0.13179, standard deviation 0.00689), `pdays` (0.03941, standard deviation 0.00201), and `previous` (0.02339, standard deviation 0.00159).

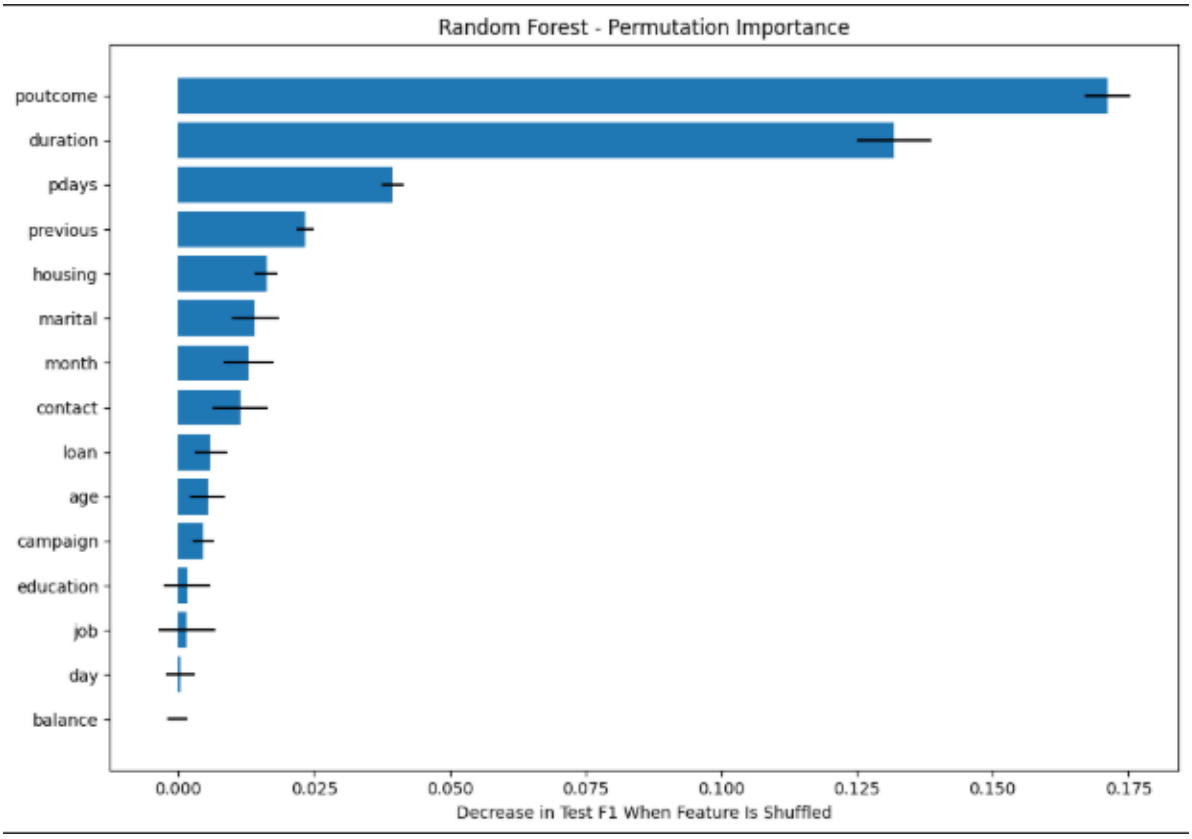


Figure 3: Permutation feature importance. The horizontal bars show the mean reduction in test F1 when a feature is shuffled, with error bars showing variation across repeats.

MDI and permutation importance both rank `duration` and previous-campaign information highly, although the ordering differs. The notebook's top three MDI features are duration, poutcome, and month; its top three permutation features are poutcome, duration, and pdays. The two approaches therefore provide complementary views of feature usefulness.

5.8 Classifier Benchmarking

The notebook compares Logistic Regression, Decision Tree, and Random Forest on the same bank-marketing task. The table below reproduces the recorded benchmark output.

Table 9: Test-set comparison across three classifiers

Model	Accuracy	Precision	Recall	F1	ROC-AUC
Logistic Regression	0.8478	0.4221	0.8162	0.5565	0.9086
Decision Tree	0.7577	0.3086	0.8638	0.4547	0.8787
Random Forest	0.9014	0.7374	0.2443	0.3670	0.9243

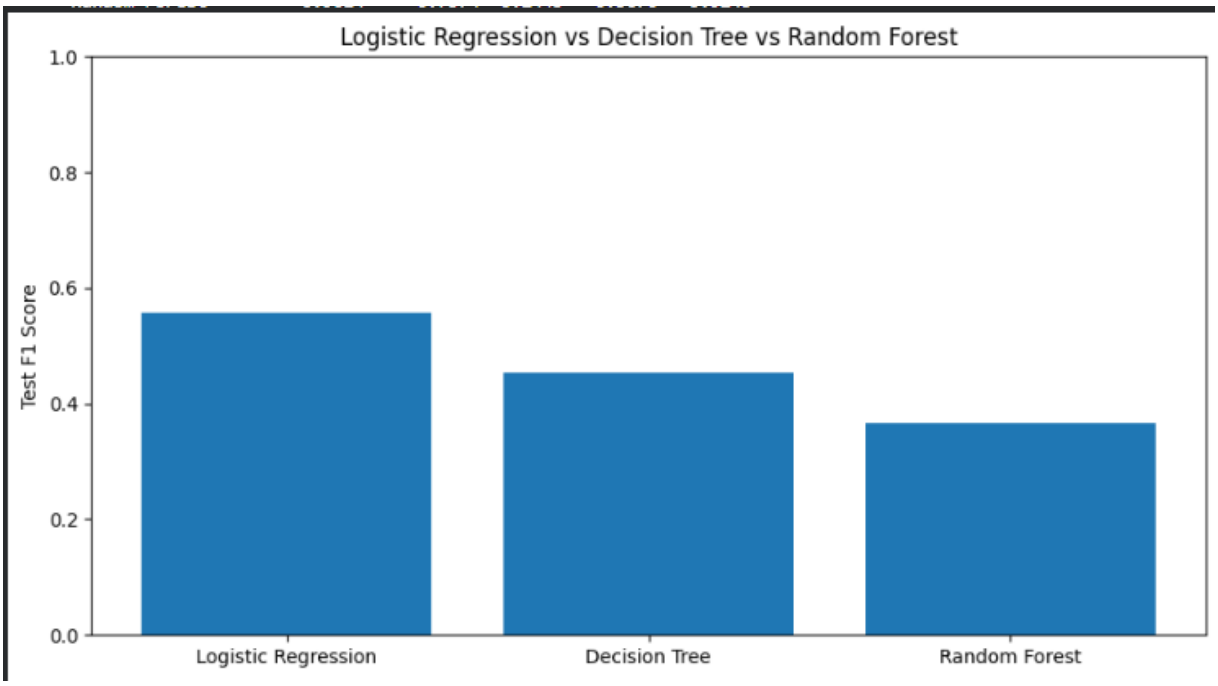


Figure 4: Test F1-score comparison for Logistic Regression, Decision Tree, and Random Forest.

The Random Forest has the highest accuracy and ROC-AUC in this benchmark. Logistic Regression has the highest F1-score and substantially higher recall than the Random Forest. Thus, the best model depends on the business objective: ranking quality, minimizing unnecessary calls, or capturing as many subscribers as possible.

5.9 Business Interpretation

The feature-importance analysis suggests that the most recent call's duration and previous campaign outcome carry substantial predictive information in this dataset. These are predictive associations rather than proof that changing a feature will cause a subscription. Campaign managers can use the model to prioritize leads, but should choose the probability threshold based on outreach capacity and the relative cost of missed subscribers and false-positive calls. Model monitoring should track minority-class recall and F1 as well as accuracy.

6 Exercise 5: K-Means – Wholesale Customer Spending Personas

6.1 Objective and Dataset

The final exercise applies unsupervised learning to group wholesale customers according to annual spending patterns. The UCI Wholesale Customers dataset has 440 rows and eight columns: channel, region, and six spending categories. Clustering uses the six numeric categories: Fresh, Milk, Grocery, Frozen, Detergents_Paper, and Delicassen. Channel and region are not included in the six-dimensional spending feature matrix used for clustering.

There are no missing values in the six spending columns. The data contains large differences in scale and strongly right-skewed distributions. For example, Fresh has a maximum of 112,151, Grocery has a maximum of 92,780, and Delicassen has a maximum of 47,943. These ranges make transformation and scaling important before applying a distance-based algorithm.

6.2 Code Walkthrough: Log Transformation and Scaling

The notebook applies $\log(1+x)$ to each spending feature, reducing the influence of very large values and compressing the scale of high-spending observations. It then applies `StandardScaler`, which centres each transformed feature around zero and scales it to unit variance. The resulting standardized feature matrix is used by K-Means.

Listing 4: Core K-Means preprocessing and fitting sequence

```
log_spending = np.log1p(spending_features)
scaled_spending = StandardScaler().fit_transform(log_spending)

for k in range(2, 9):
    model = KMeans(n_clusters=k, random_state=42, n_init=10)
    labels = model.fit_predict(scaled_spending)
    inertia_by_k[k] = model.inertia_
    silhouette_by_k[k] = silhouette_score(scaled_spending, labels)

final_model = KMeans(n_clusters=4, random_state=42, n_init=10)
cluster_labels = final_model.fit_predict(scaled_spending)
```

The Elbow method records inertia, or within-cluster sum of squared distances, for candidate values of k. Inertia normally decreases as the number of clusters increases. The silhouette score measures how similar a point is to its own cluster compared with other clusters; a higher value generally indicates better separation and cohesion.

6.3 Elbow Method

Table 10: Inertia values for candidate cluster counts

Number of clusters (k)	Inertia
2	1844.064
3	1553.421
4	1386.863
5	1266.149
6	1174.759
7	1086.359
8	1028.486

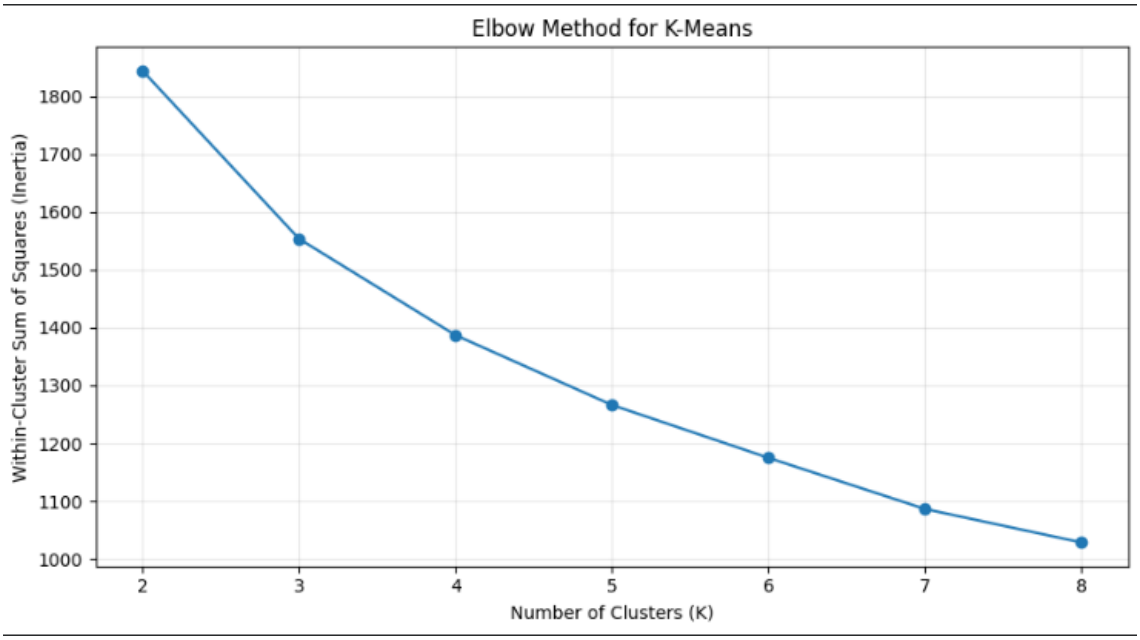


Figure 5: Elbow curve showing the within-cluster sum of squares for $k = 2$ through $k = 8$.

The curve shows a steady decrease in inertia as additional clusters are introduced. The notebook retains a four-cluster solution for the final business profiling, consistent with the lab guide's request to translate four groups into business personas.

6.4 Silhouette Analysis

Table 11: Silhouette scores for candidate cluster counts

k	**Silhouette score**
2	0.2903
3	0.2594
4	0.1885
5	0.1916
6	0.2009
7	0.1958
8	0.1852

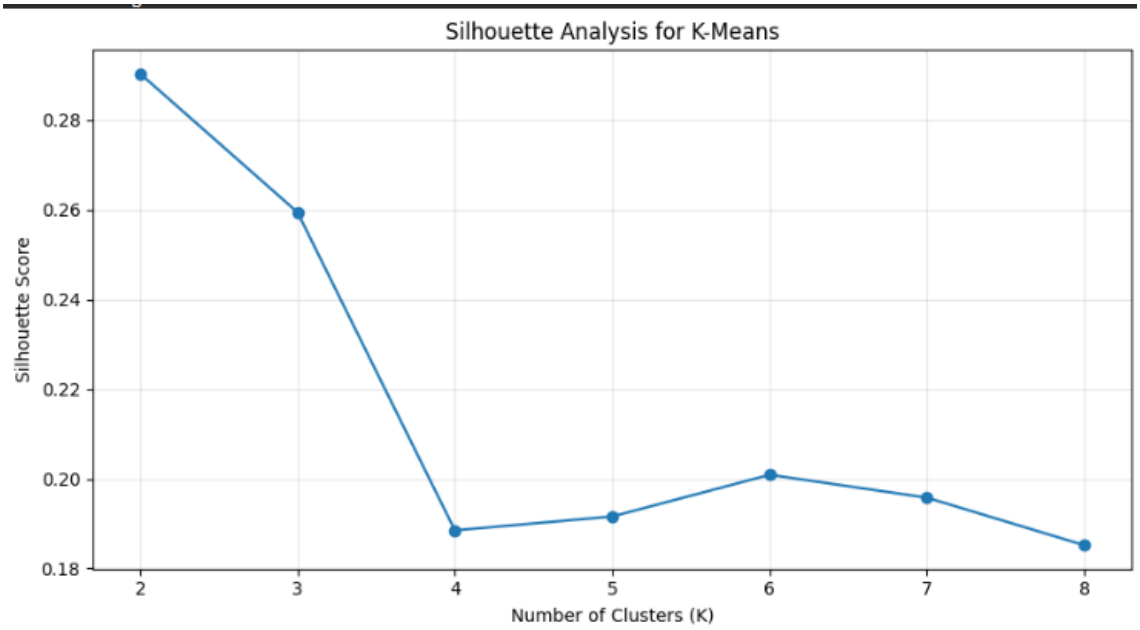


Figure 6: Silhouette analysis for candidate values of k.

The highest silhouette score among the tested values is 0.2903 at $k = 2$. The final profile analysis nevertheless uses $k = 4$ so that four spending groups can be described and compared as requested by the lab guide. The silhouette score for this selected solution is 0.1885.

6.5 Four-Cluster Spending Profiles

The final model assigns 440 customers to four clusters. Cluster sizes are 61, 129, 132, and 118 customers, respectively. The centroids below are shown in original spending units rather than standardized log units, allowing the profile values to be interpreted commercially.

Table 12: Mean annual spending by cluster in original values

Cluster	Fresh	Milk	Grocery	Frozen	Detergents/Paper	Delicassen
0	2,809.05	7,061.84	12,349.33	565.59	5,342.62	635.84
1	19,374.38	4,068.11	4,444.94	6,184.33	723.33	2,322.71
2	9,632.66	1,569.58	2,071.87	2,175.23	345.01	584.81
3	11,338.75	11,759.44	16,087.87	1,968.13	6,805.99	2,163.83

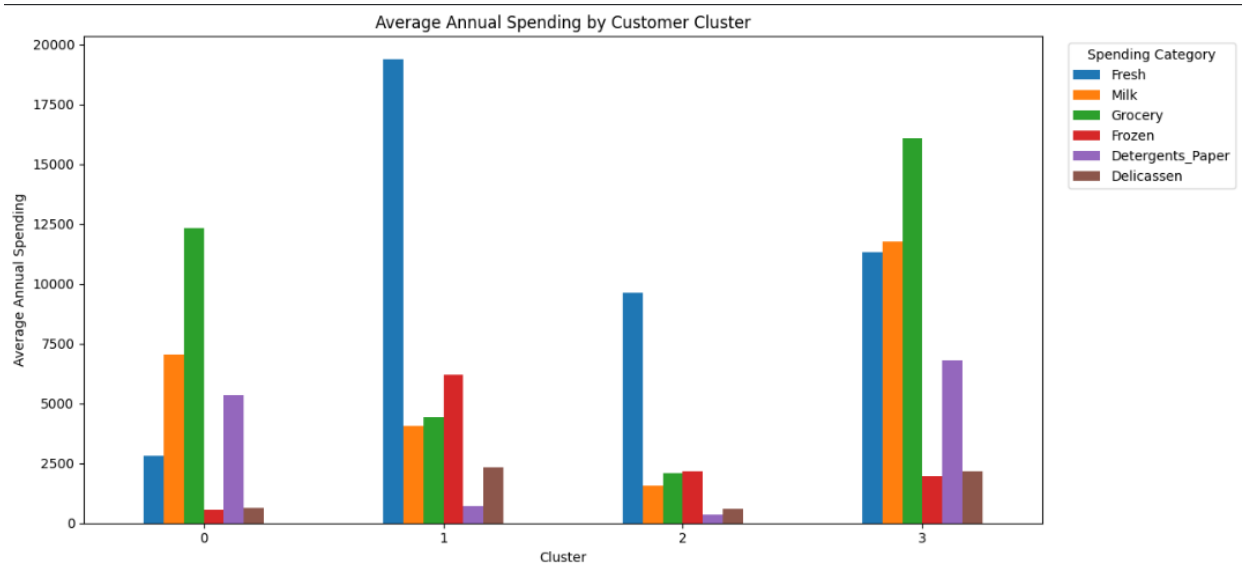


Figure 7: Average annual spending by customer cluster, plotted in original spending units.

The bar chart compares the six categories across clusters. Cluster 0 shows relatively high Grocery, Milk, and Detergents_Paper spending with low Fresh and Frozen spending. Cluster 1 has the largest mean Fresh and relatively high Frozen spending. Cluster 2 has lower average spending across most categories. Cluster 3 has high Milk, Grocery, and Detergents_Paper spending, with substantial Fresh spending as well.

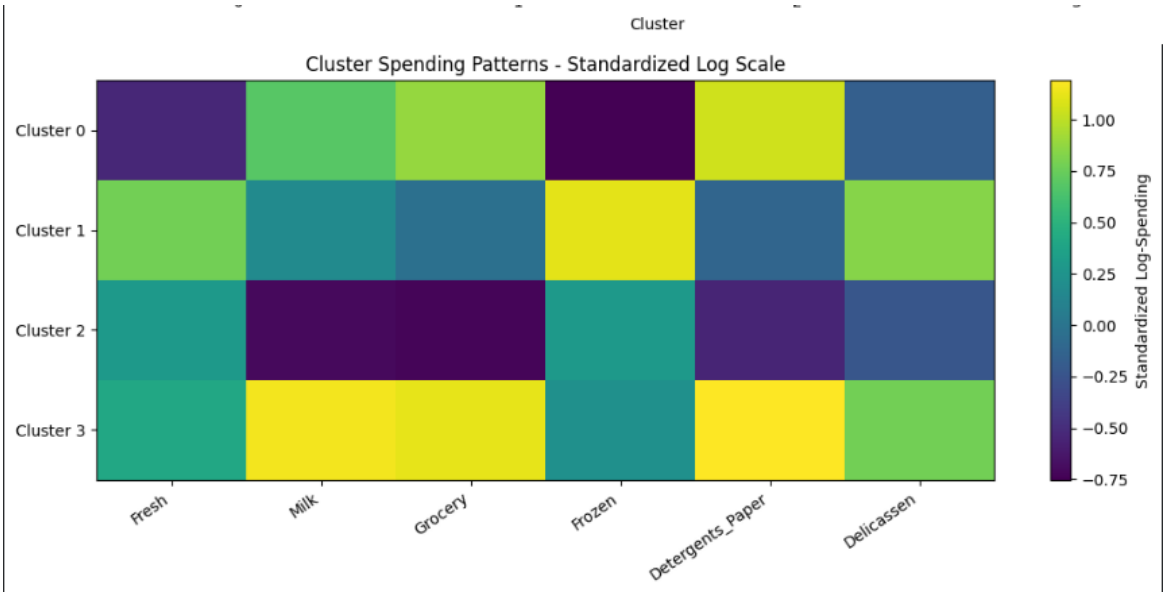


Figure 8: Heatmap of cluster spending patterns in standardized log space.

The heatmap uses standardized log-spending values, so colours indicate each cluster's relative position for a category compared with the dataset-wide standardized scale. This makes it easier to compare profile patterns across features even though their original currency values differ substantially.

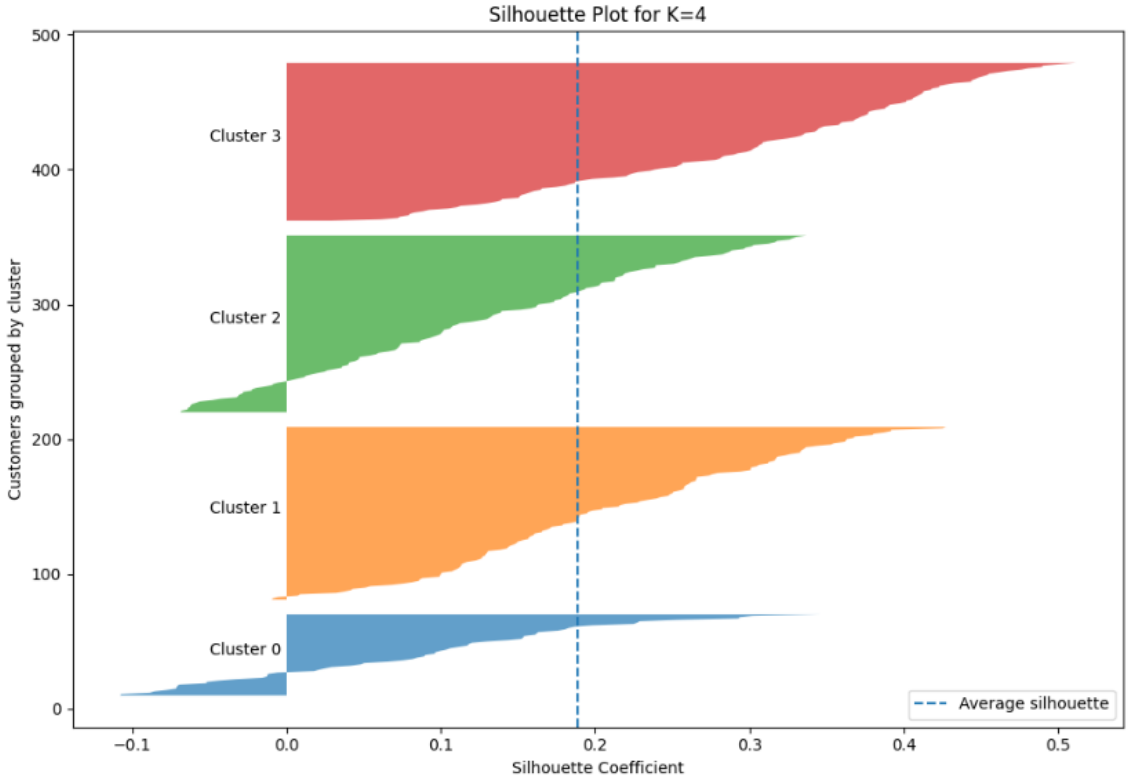


Figure 9: Silhouette plot for the final four-cluster K-Means solution.

The silhouette plot shows the distribution of silhouette coefficients within each cluster and the average silhouette reference line. It complements the overall score by showing whether observations in individual clusters are well separated or whether some observations lie near overlapping boundaries.

6.6 Customer Counts and Business Personas

Table 13: Cluster counts and personas recorded by the notebook

Cluster	Customers	Share	Persona	Profile interpretation
0	61	13.86%	Supermarkets / Grocery Retailers	Higher Grocery, Milk, and Detergents_Paper spending.
1	129	29.32%	Fresh Markets	Highest Fresh spending and elevated Frozen purchases.
2	132	30.00%	Fresh Markets	Lower average spend across most categories; a lower-volume profile.
3	118	26.82%	Boutique Delicatessens	High Milk, Grocery, and Detergents_Paper spending, plus Fresh products.

The notebook assigns suggested commercial personas to the four clusters. Clusters 1 and 2 are both labelled Fresh Markets in the recorded summary, but their spending profiles differ: Cluster 1 has substantially higher Fresh and Frozen averages, whereas Cluster 2 is lower-spending across most categories. They can therefore be treated as different purchasing tiers within the same broad persona family.

6.7 Actionable Recommendations

- **Cluster 0 – Supermarkets / Grocery Retailers:** prioritize dependable Grocery and Detergents_Paper availability, bulk replenishment, and volume-based pricing. Credit limits can be aligned with verified purchase history and stable high-volume ordering.
- **Cluster 1 – Fresh Markets:** prioritize Fresh availability, frequent deliveries, and short replenishment cycles. Credit reviews can reflect replenishment frequency and observed cash-flow patterns.
- **Cluster 2 – Lower-volume Fresh Markets:** offer smaller replenishment quantities, flexible order sizes, and product bundles suited to lower average spend. Review credit using payment history and order consistency.
- **Cluster 3 – Boutique Delicatessens:** offer specialty assortments, premium product options, and flexible order quantities. Begin with controlled credit limits and adjust them according to verified payment history.

6.8 Interpretation

K-Means provides a compact summary of customer spending patterns, but cluster labels are business interpretations rather than known ground-truth categories. The centroid table and visual profiles should be used together when designing inventory, sales, and credit strategies. The silhouette score helps indicate how cleanly the data separates into groups and should be considered alongside the practical usefulness of the segments.

7 Cross-Exercise Comparison and Reproducibility

7.1 Consolidated Results

Exercise	Main result	Diagnostic / secondary result	Interpretation
1. Regression	$R^2 = 0.7836$; RMSE \$5,796.28	Log-target RMSE \$7,814.06; smoker coefficient \$23,651.13	Raw-target fit explains much of the variation; residuals should be inspected for changing spread.
2. Logistic Regression	ROC-AUC 0.9085	Threshold 0.35 recall 0.9017; precision 0.3343	Threshold selection trades off missed subscribers against extra outreach.
3. Decision Tree	Pruned accuracy 0.9386	F1 0.9510; selected depth 3	Pruning produces a compact, interpretable tree on the notebook's dataset.
4. Random Forest	OOB 0.9005; ROC-AUC 0.9243	Test F1 0.3670; recall 0.2443	Strong ranking and accuracy coexist with lower recall for the minority class.
5. K-Means	Four clusters profiled; silhouette 0.1885	Highest tested silhouette 0.2903 at $k = 2$	Four groups support the requested business profiling; cluster separation is modest.

7.2 Saved Artifacts and Reproducibility

The notebook records output directories under `/content/exercise1_outputs` through `/content/exercise5_outp` with plots and CSV outputs in their respective subdirectories. It exports test predictions for the regression and classification exercises and cluster assignments for K-Means. When rerunning the notebook, execute the cells from top to bottom so that preprocessing objects, trained estimators, predictions, plots, and exports are created in the expected order.

For reproducibility, retain the notebook, dataset source information, and saved CSV and plot outputs together. Fixed random-state values are used in model splitting or estimator construction where specified in the notebook. The recorded numerical results in this report correspond to the supplied notebook outputs.

8 Conclusion

The five exercises demonstrate a broad machine-learning workflow, from preparing tabular datasets and building pipelines to evaluating models, interpreting diagnostics, and translating results into practical decisions. The regression exercise shows how target transformations and residual analysis affect interpretation. Logistic Regression demonstrates the importance of threshold selection when the positive class is uncommon. Decision Trees provide readable prediction paths and show the value of controlling complexity. Random Forest adds ensemble learning, OOB evaluation, hyperparameter tuning, and complementary feature-importance methods. K-Means groups wholesale customers by spending behaviour and turns cluster centroids into inventory and credit recommendations.

Across the exercises, the results show why a model should not be judged by a single metric. Regression benefits from looking at both RMSE and MAE alongside residual plots; imbalanced classification requires precision, recall, F1, ROC-AUC, and confusion matrices; tree models need both predictive evaluation and complexity control; feature importance should be interpreted as predictive association rather than causation; and clustering should be evaluated through both numerical separation and practical usefulness of the resulting profiles. Together, the notebook and this report document the implementation, results, diagnostics, and business interpretation of all five laboratory exercises.